

PART A — Basic SQL

Q1 — SELECT

Return:

id
title
file_type
page_count
from documents.

ANSWER:-

SELECT

id, title, file_type, page_count
FROM documents;

Q2 — SELECT + Alias

Return:

document_id
document_title
pages

using:

id
title
page_count

SOLUTION:-

SELECT
id AS document_id,
title AS document_title,
page_count AS pages
FROM documents;

Q3 — WHERE

Find all documents with:

page_count > 40

Return:

id

title

page_count

SOLUTION:-

SELECT

id,

title,

page_count

FROM documents

WHERE page_count > 40;

Q4 — Multiple Conditions

Find:

PDF documents that have more than 30 pages.

Return:

id

title

page_count

SOLUTION:-

SELECT

id,

title,

page_count

FROM documents

WHERE file_type = 'PDF'

AND page_count > 30;

PART B — ORDERING + LIMIT

Q5.

Return all documents ordered by:

page_count DESC

SOLUTION:-

SELECT *

FROM documents

ORDER BY page_count DESC;

Q6

Return the **3 largest documents**.

Return:

id

title

page_count

SOLUTION:-

SELECT

id,

title,

page_count

FROM documents

ORDER BY page_count DESC

LIMIT 3;

Q7

Return the **2 largest processed PDF documents**.

Requirements:

PDF

processed

largest first

2 rows

SOLUTION:-

SELECT

id,

title,

page_count

FROM documents

WHERE file_type = 'PDF'

AND status = 'processed'

ORDER BY page_count DESC

LIMIT 2;

Q8

Return unique file types.

SOLUTION:-

SELECT DISTINCT file_type

FROM documents;

Q9

Return unique countries sorted alphabetically.

SOLUTION:-

```
SELECT DISTINCT country
FROM documents
ORDER BY country ASC;
```

PART C — Aggregates

Q10

Return:

total_documents

total_pages

average_pages

minimum_pages

maximum_pages

for the entire documents table.

SOLUTION:-

```
SELECT
COUNT(*) AS total_documents,
SUM(page_count) AS total_pages,
AVG(page_count) AS average_pages,
MIN(page_count) AS minimum_pages,
MAX(page_count) AS maximum_pages
FROM documents;
```

Q11

Return the number of failed documents.

Output:

failed_documents

SOLUTION:-

```
SELECT
COUNT(*) AS failed_documents
FROM documents
WHERE status = 'failed';
```

Q12

Return the total number of pages in processed documents.

Output:

processed_pages

SOLUTION:-

SELECT

SUM(page_count) AS processed_pages

FROM documents

WHERE status = 'processed';

Q13

Return the average page count of PDF documents.

Output:

average_pdf_pages

SOLUTION:-

SELECT

AVG(page_count) AS average_pdf_pages

FROM documents

WHERE file_type = 'PDF';

PART D — GROUP BY

Q14

Count documents by status.

Return:

status

document_count

SOLUTION:-

SELECT

status,

COUNT(*) AS document_count

FROM documents

GROUP BY status;

Q15

Calculate total pages by file type.

Return:

file_type

total_pages

SOLUTION:-

SELECT

file_type,

SUM(page_count) AS total_pages

FROM documents

GROUP BY file_type;

Q16

For every user, return:

user_id

document_count

total_pages

average_pages

SOLUTION:-

SELECT

user_id,

COUNT(*) AS document_count,

SUM(page_count) AS total_pages,

AVG(page_count) AS average_pages

FROM documents

GROUP BY user_id;

Q17

For every status, return:

status

document_count

total_pages

average_pages

Order by:

document_count DESC

SOLUTION:-

```
SELECT
status,
COUNT(*) AS document_count,
SUM(page_count) AS total_pages,
AVG(page_count) AS average_pages
FROM documents
GROUP BY status
ORDER BY document_count DESC;
```

PART E — HAVING

Q18

Find users who have uploaded **at least 2 documents**.

Return:

user_id

document_count

SOLUTION:-

```
SELECT
user_id,
COUNT(*) AS document_count
FROM documents
GROUP BY user_id
HAVING COUNT(*) >= 2;
```

Q19

Find users whose total uploaded pages are **greater than 50**.

Return:

user_id

total_pages

SOLUTION:-

```
SELECT
user_id,
SUM(page_count) AS total_pages
FROM documents
GROUP BY user_id
HAVING SUM(page_count) > 50;
```

Q20

Find file types having **at least 2 documents**.

Return:

file_type

document_count

SOLUTION:-

SELECT

file_type,

COUNT(*) AS document_count

FROM documents

GROUP BY file_type

HAVING COUNT(*) >= 2;

Q21

Find users having:

at least 2 documents

AND

more than 50 total pages

Return:

user_id

document_count

total_pages

SOLUTION:-

SELECT

user_id,

COUNT(*) AS document_count,

SUM(page_count) AS total_pages

FROM documents

GROUP BY user_id

HAVING COUNT(*) >= 2

AND SUM(page_count) > 50;

PART F — CASE

Q22

Classify every document:

page_count < 20 → Small

20–40 → Medium

> 40 → Large

Return:

id

title

page_count

size_category

SOLUTION:-

SELECT

id,

title,

page_count,

CASE

WHEN page_count < 20 THEN 'Small'

WHEN page_count BETWEEN 20 AND 40 THEN 'Medium'

WHEN page_count > 40 THEN 'Large'

END AS size_category

FROM documents;

Q23

Create a pipeline_state:

processed → Healthy

processing → Running

failed → Attention

Return:

id

title

status

pipeline_state

SOLUTION:-

SELECT

```
id,  
title,  
status,  
CASE  
WHEN status = 'processed' THEN 'Healthy'  
WHEN status = 'processing' THEN 'Running'  
WHEN status = 'failed' THEN 'Attention'  
END AS pipeline_state  
FROM documents;
```

Q24

Create **both** classifications:

size_category

pipeline_state

Return:

id

title

page_count

status

size_category

pipeline_state

SOLUTION:-

SELECT

id,

title,

page_count,

status,

CASE

WHEN page_count < 20 THEN 'Small'

WHEN page_count BETWEEN 20 AND 40 THEN 'Medium'

WHEN page_count > 40 THEN 'Large'

END AS size_category,

CASE

WHEN status = 'processed' THEN 'Healthy'

WHEN status = 'processing' THEN 'Running'

```
WHEN status = 'failed' THEN 'Attention'  
END AS pipeline_state  
FROM documents;
```

PART G — Combined SQL

Now the difficulty increases.

Q25

Find the **top 3 users by total document pages**.

Return:

user_id

document_count

total_pages

Requirements:

- GROUP BY
- SUM
- COUNT
- ORDER BY
- LIMIT

SOLUTION:-

SELECT

user_id,

COUNT(*) AS document_count,

SUM(page_count) AS total_pages

FROM documents

GROUP BY user_id

ORDER BY total_pages DESC

LIMIT 3;

Q26

Find users who have:

at least 2 documents

and return them ordered by:

total_pages DESC

Return:

user_id

document_count
total_pages
SOLUTION:-
SELECT
user_id,
COUNT(*) AS document_count,
SUM(page_count) AS total_pages
FROM documents
GROUP BY user_id
HAVING COUNT(*) >= 2
ORDER BY total_pages DESC;

Q27

Find the file types where:

average page count > 25

Return:

file_type
average_pages

SOLUTION:-

SELECT
file_type,
AVG(page_count) AS average_pages
FROM documents
GROUP BY file_type
HAVING AVG(page_count) > 25;

Q28

Find statuses where:

total pages > 50

Return:

status
total_pages

SOLUTION:-

SELECT
status,
SUM(page_count) AS total_pages
FROM documents

GROUP BY status

HAVING SUM(page_count) > 50;

PART H — SQL Debugging

These are intentionally broken.

Your job isn't just to fix them.

Explain **why** they're broken.

Q29

SELECT

title,

page_count

FROM documents

LIMIT 3

ORDER BY page_count DESC;

Find the problem and correct it.

SOLUTION:-

SELECT

title,

page_count

FROM documents

ORDER BY page_count DESC

LIMIT 3;

Q30

SELECT

status,

COUNT(*) AS document_count

FROM documents

WHERE COUNT(*) >= 2

GROUP BY status;

Find the problem and correct it.

SOLUTION:-

SELECT

status,

COUNT(*) AS document_count

FROM documents

GROUP BY status

HAVING COUNT(*) >= 2;

Q31

SELECT

user_id,

status,

COUNT(*) AS document_count

FROM documents

GROUP BY user_id;

Why is this invalid?

Correct it.

SOLUTION:-

SELECT

user_id,

status,

COUNT(*) AS document_count

FROM documents

GROUP BY user_id, status;

Q32

SELECT

title,

CASE

WHEN page_count > 20 THEN 'Medium'

WHEN page_count > 40 THEN 'Large'

ELSE 'Small'

END AS size_category

FROM documents;

There is a **logic bug**, not a syntax bug.

Explain it and correct it.

SOLUTION:-

SELECT

title,

```
CASE
WHEN page_count > 40 THEN 'Large'
WHEN page_count >= 20 THEN 'Medium'
ELSE 'Small'
END AS size_category
FROM documents;
```

PART I — MINI PROJECT

Now you're going to solve a small real-world Data Engineering problem.

Scenario

You're working on a RAG platform.

The backend team wants an **ingestion analytics query** for the admin dashboard.

The dashboard needs to show:

Per user:

user_id
document_count
total_pages
average_pages
processed_documents
failed_documents
size_category

Where size_category represents the user's **overall document volume**:

total_pages < 50 → Low

50–100 → Medium

> 100 → High

And the dashboard should:

- Include only users with at least 1 document
- Sort users by total_pages DESC
- Return the top 5 users

Mini Project Requirements

You need to combine:

GROUP BY

COUNT

SUM

AVG

CASE

ORDER BY

LIMIT

For:

processed_documents

failed_documents

use **conditional aggregation**.

Expected output structure

user_id

document_count

total_pages

average_pages

processed_documents

failed_documents

size_category

Write **one SQL query**.

Important

Don't create separate queries for each metric.

Think about how you can calculate all metrics for each user_id in a single grouped query.

SOLUTION:-

SELECT

user_id,

COUNT(*) AS document_count,

SUM(page_count) AS total_pages,

AVG(page_count) AS average_pages,

SUM(

CASE

WHEN status = 'processed' THEN 1

ELSE 0

END

) AS processed_documents,

SUM(

CASE

```

WHEN status = 'failed' THEN 1
ELSE 0
END
) AS failed_documents,

CASE
WHEN SUM(page_count) < 50 THEN 'Low'
WHEN SUM(page_count) BETWEEN 50 AND 100 THEN 'Medium'
ELSE 'High'
END AS size_category

FROM documents
GROUP BY user_id
ORDER BY total_pages DESC
LIMIT 5;

```

How the query works

Requirement	SQL implementation
Per-user grouping	GROUP BY user_id
Total documents	COUNT(*)
Total pages	SUM(page_count)
Average pages	AVG(page_count)
Processed documents	Conditional SUM(CASE WHEN status = 'processed'...)
Failed documents	Conditional SUM(CASE WHEN status = 'failed'...)
Volume classification	CASE based on SUM(page_count)
Largest users first	ORDER BY total_pages DESC
Top 5 users	LIMIT 5

Important: The requirement “at least 1 document” is already satisfied because GROUP BY user_id creates groups only from users who have rows in documents. No separate WHERE condition is necessary.

Conditional aggregation pattern to remember:

```

SUM(
CASE
WHEN condition THEN 1
ELSE 0
END
)

```

This counts only the rows that satisfy the condition within each group.

MINI PROJECT — Business Explanation

1 . What does one output row represent?

One output row represents one user and their document ingestion statistics.

For example, a row might show that user U 1 0 1 has uploaded 8 documents, totaling 1 2 0 pages, with 6 processed and 2 failed.

2 . What does GROUP BY user_id accomplish?

GROUP BY user_id combines all documents belonging to the same user into a single group. This allows aggregate functions such as COUNT(), SUM(), and AVG() to calculate metrics separately for each user, rather than for the entire table.

3 . How are processed_documents and failed_documents calculated?

They use conditional aggregation.

For each document, the CASE expression returns 1 if the status matches and 0 otherwise. SUM() then adds those values within each user's group.

```
SUM(CASE WHEN status = 'processed' THEN 1 ELSE 0 END)
```

- processed → counted in processed_documents
- failed → counted in failed_documents
- Other statuses → contribute 0

4 . Why does CASE belong in the SELECT?

The CASE expression creates a derived output column, size_category, based on the user's aggregated total_pages.

It belongs in SELECT because the dashboard needs to display the classification for each user.

```
CASE
```

```
WHEN SUM(page_count) < 50 THEN 'Low'
```

```
WHEN SUM(page_count) BETWEEN 50 AND 100 THEN 'Medium'
```

```
ELSE 'High'
```

```
END AS size_category
```

5 . Why is ORDER BY applied after the aggregation?

The dashboard needs to rank users by their total pages, not individual documents.

First, GROUP BY and SUM(page_count) calculate each user's total. Then ORDER BY total_pages DESC sorts those aggregated user-level results from largest to smallest.

6 . Why is LIMIT 5 useful for the dashboard?

LIMIT 5 returns only the top 5 users with the highest total page volume.

This is useful for a dashboard because it:

- Keeps the initial view focused on the highest-volume users.
- Reduces the number of rows displayed.
- Makes the query result more efficient for a “Top Users” widget.
- Helps administrators quickly identify users generating the most ingestion workload.

Overall business meaning: This query transforms document-level ingestion data into a user-level analytics report that helps administrators understand document volume, processing health, and workload concentration.

MINI PROJECT DEBUGGING

Now imagine another engineer wrote:

```
SELECT
user_id,
COUNT(*) AS document_count,
SUM(page_count) AS total_pages,
AVG(page_count) AS average_pages,
CASE
WHEN SUM(page_count) > 50 THEN 'Medium'
WHEN SUM(page_count) > 100 THEN 'High'
ELSE 'Low'
END AS size_category
FROM documents
GROUP BY user_id
HAVING COUNT(*) >= 1
ORDER BY total_pages DESC
LIMIT 5;
```

There is a **logic bug** in the CASE.

Your task

Explain:

1. What happens to a user with 150 pages?
2. Why?
3. How should the CASE conditions be ordered?

Don't just rewrite it—explain the reasoning.

SOLUTION:-

1. What happens to a user with 150 pages?

A user with 150 total pages is incorrectly classified as Medium.

2. Why?

The CASE expression evaluates conditions from top to bottom and stops at the first condition that is true.

For a user with 150 pages:

$SUM(page_count) > 50 \rightarrow TRUE$

Because this condition is already true, SQL returns 'Medium' and never evaluates the next condition:

```
WHEN SUM(page_count) > 100 THEN 'High'
```

Therefore, even though 150 is greater than 100, the user is classified incorrectly.

3. How should the CASE conditions be ordered?

The more restrictive, higher-threshold condition must come first:

```
CASE
```

```
WHEN SUM(page_count) > 100 THEN 'High'
```

```
WHEN SUM(page_count) >= 50 THEN 'Medium'
```

```
ELSE 'Low'
```

```
END AS size_category
```

Why this ordering works

Total pages	First condition (> 100)	Second condition (>= 50)	Result
30	False	False	Low
50	False	True	Medium
100	False	True	Medium
101	True	—	High
150	True	—	High

Interview takeaway: CASE uses first-match evaluation. When conditions overlap, place the more specific or higher-priority condition first so broader conditions do not capture those rows prematurely.

PART J — Interview Assessment

Here are interview-ready answers you can speak naturally in a Junior Data Engineer / GenAI Engineer interview.

Q33 — Difference Between WHERE and HAVING

Interview answer:

WHERE filters individual rows before grouping or aggregation, whereas HAVING filters groups after aggregation. For example, I would use WHERE status = 'processed' to filter processed documents before calculating metrics. If I want users with more than 50 total pages, I would use HAVING SUM(page_count) > 50 because that condition depends on an aggregate result.

Example:

```
SELECT
```

```
user_id,
```

```
SUM(page_count) AS total_pages
```

```
FROM documents
```

```
WHERE status = 'processed'
```

```
GROUP BY user_id
```

```
HAVING SUM(page_count) > 50;
```

Q34 — Explain GROUP BY user_id

Interview answer:

GROUP BY user_id groups all document records belonging to the same user into one logical group. This allows aggregate functions like COUNT, SUM, and AVG to calculate metrics independently for each user. For example, in an ingestion analytics dashboard, I can use it to find how many documents each user uploaded and how many total pages they contributed.

Example:

```
SELECT
user_id,
COUNT(*) AS document_count
FROM documents
GROUP BY user_id;
```

Q35 — Difference Between COUNT(*) and COUNT(page_count)

Interview answer:

COUNT(*) counts every row returned by the query, including rows where page_count is NULL. COUNT(page_count) counts only rows where page_count is not NULL. So, if I need the total number of document records, I use COUNT(*). If I specifically need the number of documents with a known page count, I use COUNT(page_count).

Example:

id	page_count
1	20
2	NULL
3	40

```
SELECT
```

```
COUNT(*) AS all_rows,
```

```
COUNT(page_count) AS known_page_counts
```

```
FROM documents;
```

Result:

all_rows	known_page_counts
3	2

Q36 — Why Not Depend on LIMIT 10 Without ORDER BY in an API?

Interview answer:

Without ORDER BY, SQL does not guarantee which 10 rows will be returned. The database may return different rows depending on the execution plan, indexes, or data changes. For an API, this can lead to inconsistent results and unreliable pagination. I would always define an explicit ordering, such as ORDER BY created_at DESC, id DESC, depending on the business requirement.

Better example:

```
SELECT
```

```
id,  
title,  
created_at  
    FROM documents  
    ORDER BY created_at DESC, id DESC  
LIMIT 10;
```

Engineering point: If implementing pagination, use a stable, deterministic sort, ideally with a unique tie-breaker such as id.

Q37 — Explain CASE WHEN Using a Real Engineering Example

Interview answer:

CASE WHEN is SQL's conditional expression. I use it to derive business or engineering classifications directly in a query. For example, in a RAG ingestion pipeline, I can classify documents based on their processing status: processed documents are healthy, processing documents are running, and failed documents need attention. This avoids requiring separate application-side logic for a simple classification.

Example:

```
SELECT  
id,  
title,  
status,  
CASE  
    WHEN status = 'processed' THEN 'Healthy'  
    WHEN status = 'processing' THEN 'Running'  
    WHEN status = 'failed' THEN 'Attention'  
    ELSE 'Unknown'  
END AS pipeline_state  
    FROM documents;
```

Q38 — What Is Wrong With This Query?

```
SELECT  
user_id,  
status,  
COUNT(*)  
    FROM documents  
    GROUP BY user_id;
```

Interview answer:

The query selects status, but it only groups by user_id. Since a user can have multiple statuses, the database cannot determine which status should be returned for each user. In standard SQL,

this is invalid because every selected column must either be included in the GROUP BY or be aggregated. To fix it, I would group by both user_id and status if I want counts for each user-status combination.

Correct query:

```
SELECT
user_id,
status,
COUNT(*) AS document_count
FROM documents
GROUP BY user_id, status;
```

Alternative: If the requirement is one row per user, then status must be removed or transformed using an appropriate aggregate or business rule.

Quick Interview Revision

Concept	Key point
WHERE	Filters rows before aggregation
HAVING	Filters groups after aggregation
GROUP BY	Creates groups for aggregate calculations
COUNT(*)	Counts all rows
COUNT(column)	Counts non-NULL values
ORDER BY	Guarantees the requested sort order
CASE WHEN	Implements conditional logic
GROUP BY validity	Selected non-aggregated columns must be grouped

PART K — Explanation Challenge

Without focusing on individual keywords, explain this query in **business terms**:

```
SELECT
user_id,
COUNT(*) AS document_count,
SUM(page_count) AS total_pages,
AVG(page_count) AS average_pages,
CASE
WHEN SUM(page_count) > 100 THEN 'Heavy'
WHEN SUM(page_count) > 50 THEN 'Medium'
ELSE 'Light'
END AS usage_level
FROM documents
WHERE status = 'processed'
```

```
GROUP BY user_id
HAVING COUNT(*) >= 2
ORDER BY total_pages DESC
```

LIMIT 5;

Your explanation must cover:

- Which documents are considered?
- What does one row represent?
- What calculations are performed?
- What users are removed?
- How are results sorted?

Why are only 5 returned?

SOLUTION:-

1. Which documents are considered?

Only documents whose processing status is processed are considered.

Failed, processing, or other-status documents are excluded before the user-level calculations.

2. What does one row represent?

One row represents one user and their processed-document usage statistics.

For example, a row could represent User 101's total processed documents, total pages, and average pages per document.

3. What calculations are performed?

For each user, the query calculates:

document_count → Number of processed documents.

total_pages → Total pages across those processed documents.

average_pages → Average page count per processed document.

usage_level → Classifies the user's processed-page volume:

More than 100 pages → Heavy

More than 50 pages → Medium

50 pages or fewer → Light

4. What users are removed?

Users with fewer than 2 processed documents are removed.

The query is not checking total documents across all statuses. It checks the count after filtering to processed documents.

For example:

User	Processed docs	Failed docs	Included?
A	3	2	Yes
B	1	5	No
C	0	4	No

5. How are results sorted?

The remaining users are sorted by total_pages DESC, meaning users with the highest total number of processed pages appear first.

This prioritizes the users generating the greatest processed-document volume.

6. Why are only 5 returned?

LIMIT 5 returns only the top 5 users after filtering and sorting.

For a dashboard, this provides a focused view of the highest-usage users instead of displaying every qualifying user.

PART L — Data Engineering Thinking

Imagine your production RAG system has:

50 million documents

Your dashboard executes:

```
SELECT
user_id,
COUNT(*),
SUM(page_count)
FROM documents
GROUP BY user_id;
```

The result may contain only:

100,000 users

Question

Why can this still be an expensive query?

Give at least **3 reasons**.

Think about:

data scanning

grouping

memory

CPU

sorting/hashing

indexes

You don't need to know PostgreSQL internals yet.

SOLUTION:-

Thinking

Interview-Ready Answer

Even if the query returns only 100,000 users, it can still be expensive because the database must process the underlying 50 million document records to calculate the aggregates. The cost of a query depends not only on the size of the result, but also on the amount of data scanned and the work required to produce that result.

At Least 3 Reasons

1. Data Scanning — 50 Million Rows

The database may need to scan a large portion of the documents table to read `user_id` and `page_count` for every document.

Even though the output contains only 100,000 users, the input contains 50 million rows.

Key point: Small result $\neq$ small processing cost.

2. Grouping — Building User-Level Aggregates

The database must combine 50 million document records into groups based on `user_id`.

It needs to maintain aggregate state for each user, such as:

- Document count
- Total pages

Grouping can require substantial CPU work, especially when many rows belong to different users.

3. Memory — Maintaining Aggregate State

The database may need memory to maintain intermediate aggregation results for up to 100,000 user groups.

Depending on the execution strategy, it may use a hash table or another structure to track counts and sums. If the required memory exceeds available working memory, intermediate data may spill to disk, increasing execution time.

4. CPU — Counting and Summing

For each document, the database must perform aggregation work:

Read row → Identify user → Increment count → Add `page_count`

Repeating this across 50 million rows can consume significant CPU, even though the final output is relatively small.

5. Sorting or Hashing — Grouping Strategy

The database may use hash aggregation or sort-based aggregation.

- Hash aggregation: Builds a hash structure keyed by `user_id`.
- Sort-based aggregation: May sort rows by `user_id` before combining them.

Both approaches require computational resources, and sorting can be particularly expensive for large datasets.

6. Indexes — Helpful, but Not a Guarantee

An index on `user_id` may help in some execution plans, but it does not automatically make this query cheap.

The database still needs to read and aggregate `page_count` across the relevant documents. If the query needs a large portion of the table, a sequential scan may be more efficient than repeatedly accessing table rows through an index.

Production Perspective

For a production RAG platform, if this dashboard query runs frequently, I would consider:

- Pre-aggregated analytics tables or summary tables.
- Materialized views refreshed on a schedule.
- Incremental aggregation during document ingestion.
- Partitioning where appropriate, especially for time-based analytics.
- Query execution analysis using tools such as `EXPLAIN` / `EXPLAIN ANALYZE`.
- Caching for frequently requested dashboard metrics.

Strong Closing Statement

“The main reason is that aggregation cost is driven by the volume of input data and the work required to group it, not just the number of output rows. With 50 million documents, scanning, hashing or sorting, maintaining aggregate state, and CPU processing can all be expensive. For a frequently accessed production dashboard, I would evaluate pre-aggregation, incremental updates, and query plans rather than relying only on the raw aggregation query.”

PART M — FINAL DAY 7 TEST

Q39 — All Active Users from India

Assuming the `users` table contains `status` and `country` columns:

```
SELECT *  
FROM users  
WHERE status = 'active'  
AND country = 'India';
```

Q40 — 5 Largest PDF Documents

```
SELECT  
id,  
title,  
page_count
```

```
FROM documents
WHERE file_type = 'PDF'
ORDER BY page_count DESC
LIMIT 5;
```

Q41 — Count Documents by file_type

```
SELECT
file_type,
COUNT(*) AS document_count
FROM documents
GROUP BY file_type;
```

Q42 — Users with at Least 2 Documents

```
SELECT
user_id,
COUNT(*) AS document_count
FROM documents
GROUP BY user_id
HAVING COUNT(*) >= 2;
```

Q43 — File Types with Average Page Count Exceeding 30

```
SELECT
file_type,
AVG(page_count) AS average_pages
FROM documents
GROUP BY file_type
HAVING AVG(page_count) > 30;
```

Q44 — Label Documents by Pipeline Status

```
SELECT
id,
title,
status,
CASE
WHEN status = 'failed' THEN 'Attention'
WHEN status = 'processing' THEN 'Running'
WHEN status = 'processed' THEN 'Healthy'
ELSE 'Unknown'
END AS pipeline_state
```

FROM documents;

Q45 — Overall Document and Processing Metrics

Use conditional aggregation to calculate all metrics in one query:

```
SELECT
COUNT(*) AS total_documents,
SUM(
CASE
WHEN status = 'processed' THEN 1
ELSE 0
END
) AS processed_documents,
SUM(
CASE
WHEN status = 'failed' THEN 1
ELSE 0
END
) AS failed_documents,
SUM(page_count) AS total_pages
FROM documents;
```

Q46 — Biggest Conceptual Difference

Interview-style answer:

WHERE, GROUP BY, HAVING, and CASE solve different problems in a SQL query. WHERE decides which individual rows should be included. GROUP BY combines those rows into groups so I can calculate metrics for each group. HAVING filters the groups after aggregation. CASE is different from all three because it doesn't filter or group data—it derives a new value based on conditions.

SQL concept	Main purpose	Business question
WHERE	Filter individual rows	Which documents should I consider?
GROUP BY	Create groups	How should I organize the data for analysis?
HAVING	Filter aggregated groups	Which users or categories meet my metric threshold?
CASE	Derive conditional values	How should I classify or label each result?

Easy way to remember:

WHERE → Which rows?

GROUP BY → Which groups?

HAVING → Which groups qualify?

CASE → What label/value should I produce?